

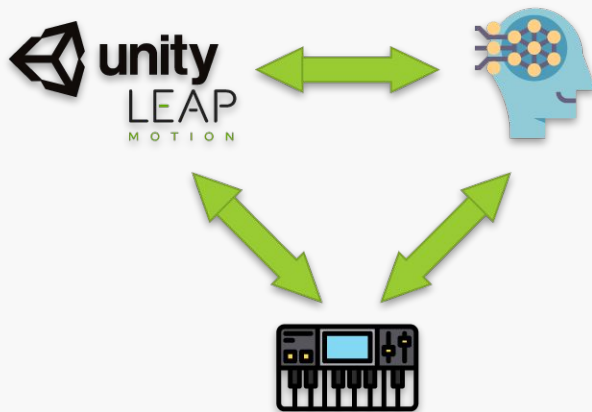
MarcoSmiles

Human-Computer Interaction & Music:
Sintetizzatore Sonoro in Unity

Overview del Sistema

In fase di progettazione, abbiamo diviso il sistema in 3 *moduli*:

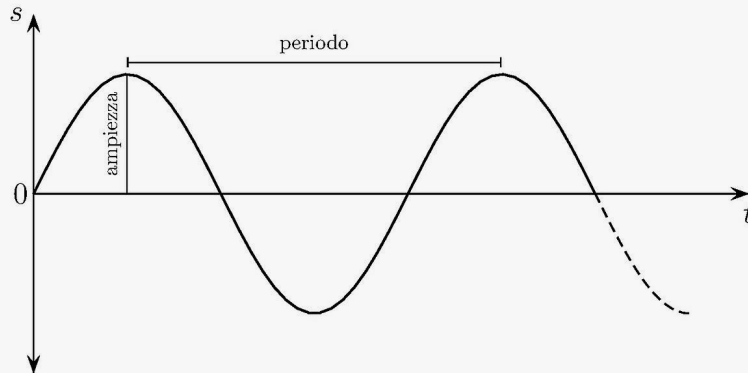
- **Modulo Unity & LeapMotion:** interfacciamento Leap Motion con Unity3D;
- **Modulo ML:** sviluppo ed utilizzo di un machine learning classifier;
- **Modulo Sonoro:** generazione del suono in real-time.



Fondamenti di Acustica

Le onde sonore possono essere rappresentate graficamente utilizzando un grafico cartesiano.

- Il **periodo** è il tempo impiegato dalla particella per tornare nello stesso punto dopo aver cominciato lo spostamento (la durata di un'oscillazione completa).
- La distanza dalla cresta all'asse delle ascisse indica l'**ampiezza** del movimento.
- Il numero di periodi compiuti in un secondo esprime la **frequenza** in hertz (Hz).
- La **Fase** φ (phi) è un parametro che indica la posizione in cui si trova il ciclo dell'onda in un determinato istante. Si misura in gradi (0° , 90° , 180° ...) o radianti.



Fondamenti di Digital Audio

Per convertire un'onda sonora in un segnale digitale, il computer effettua misurazioni dell'**ampiezza** in determinati punti. Ogni misura rilevata viene chiamata **campione** (o sample).

- La **frequenza di campionamento** è il numero di misurazioni fatte al secondo per convertire un segnale analogico in digitale.
- La **profondità di bit** permette di definire il range dinamico del segnale.
- La **codifica digitale PCM** è il nome generico per il sistema di registrazione digitale. Useremo il termine "**Codifiche PCM**" per riferirci ad una sequenza di campionamenti digitali che descrivono un segnale analogico.

Il teorema del campionamento di **Nyquist-Shannon** afferma che, se la frequenza di campionamento è sufficientemente grande, non si hanno perdite di informazione rispetto alla forma d'onda originale.

Audio in Unity & OnAudioFilterRead()

- In Unity, i suoni sono generati da componenti **Audio Source**.
- I suoni emessi vengono poi percepiti da una componente **Audio Listener**.
- Componenti **Audio Filter** permettono di modificare l'output di componenti *Audio Source* o *Audio Listener*.
- **MonoBehaviour.OnAudioFilterRead(float[] data, int channels)** : viene chiamato ogni volta che un oggetto AudioSource cerca di riprodurre un suono.

Il parametro **data** contiene i **campionamenti PCM** che l'oggetto AudioSource riproduce.

Il parametro **channels** rappresenta il numero di canali disponibili che riproducono il suono.



Modulo Sonoro: Obiettivi

Il **motore audio** e il sintetizzatore sonoro sono stati sviluppati avendo in mente i seguenti obiettivi:

- **Reattività** nel riprodurre suoni, **senza appesantire il sistema** finale;
- Elevata **portabilità**, sia del motore audio stesso, che del sistema nella propria interezza;
- Possibilità di **personalizzazione** del suono da parte dell'Utente.



Modulo Sonoro: Ipotesi Implementative

Analizzando **Unity Engine**, abbiamo ipotizzato tre possibili modi di generare il suono:

- *Triggers* di **clip audio**;
- Interfacciamento con la tecnologia **MIDI**;
- Generazione del suono in modo procedurale, utilizzando **codifiche PCM**.

La generazione del **suono in modo procedurale** è apparsa essere la soluzione più adatta per il sistema, in quanto permette di ottimizzare l'utilizzo computazionale, mantenendo una forte reattività da parte del nostro sistema.

Inoltre, questo metodo risolve eventuali **problemi di sincronizzazione** dei suoni.

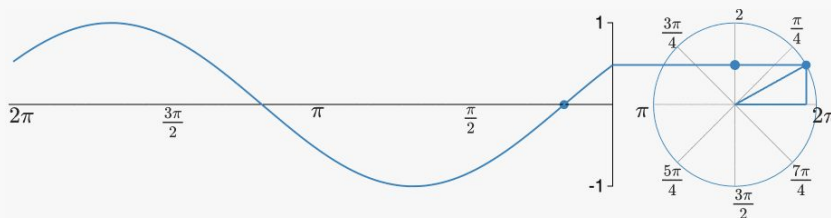
Un primo approccio: Funzioni Periodiche

Possiamo calcolare campionamenti di audio digitale che descrivono un segnale ed inoltrarli al motore audio di Unity usando **OnAudioFilterRead**.

Per farlo, incrementiamo la fase del segnale in modo costante e, ad ogni incremento, calcoliamo un nuovo campionamento, usando la fase corrente del segnale ed una funzione che descriva una funzione periodica, come **Mathf.Sin()**.

Se la fase del segnale diventa maggiore di 2π , la resettiamo, assicurandoci che la fase sia in **modulo 2π** .

$$\text{phaseIncrement} = 2.0 * \text{Mathf.PI} * \text{frequency} / \text{sampleRate}$$



Un primo approccio: Funzioni Periodiche

- **Square Wave:**

$$\begin{cases} -1 & \text{se } -\frac{T}{2} \leq t < 0 \\ 1 & \text{se } 0 \leq t \leq \frac{T}{2} \end{cases}$$



- **Sawtooth Wave:**

amplitude - (amplitude / $\text{Mathf.PI} * \text{phase}$)



Combinare Segnali Digitali

Come combinare assieme campionamenti di più segnali?

Con l'audio digitale abbiamo un **limitato range** di dinamicità (ampiezza), che dipende dalla profondità di bit in utilizzo.

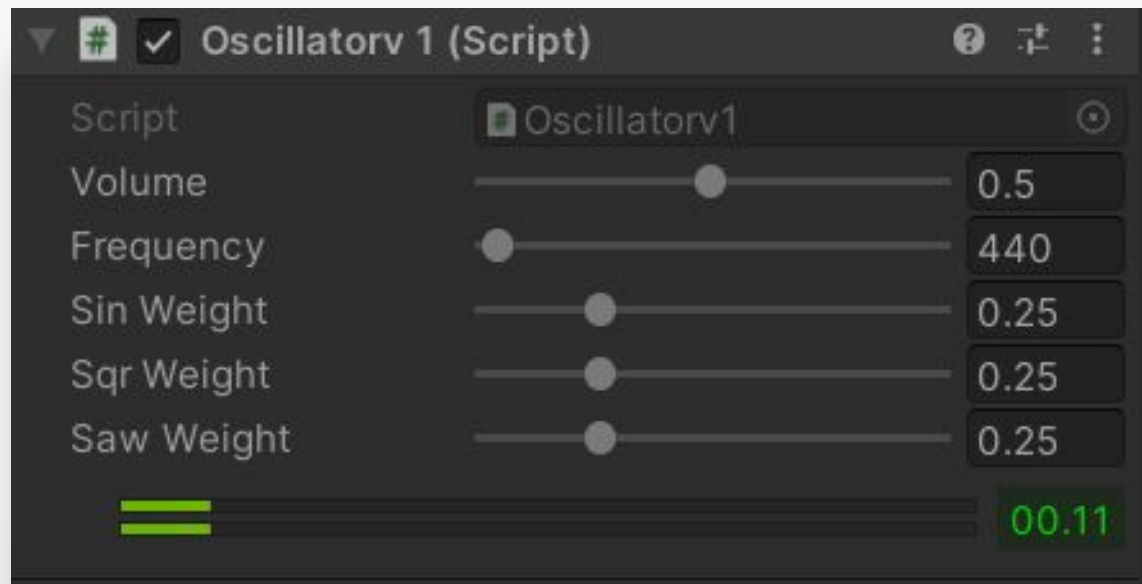
Non possiamo limitarci a sommare i campionamenti tra di loro, perché il risultato potrebbe essere un numero che va al di fuori di questo **range**.

- Possiamo usare la seguente equazione per combinare due segnali **A** e **B**:

$$Z = A + B - AB$$

- Se aggiungiamo un altro segnale **C** a **Z**, otteniamo:

$$T = Z + C - ZC = A + B + C - AB - AC - BC + ABC$$



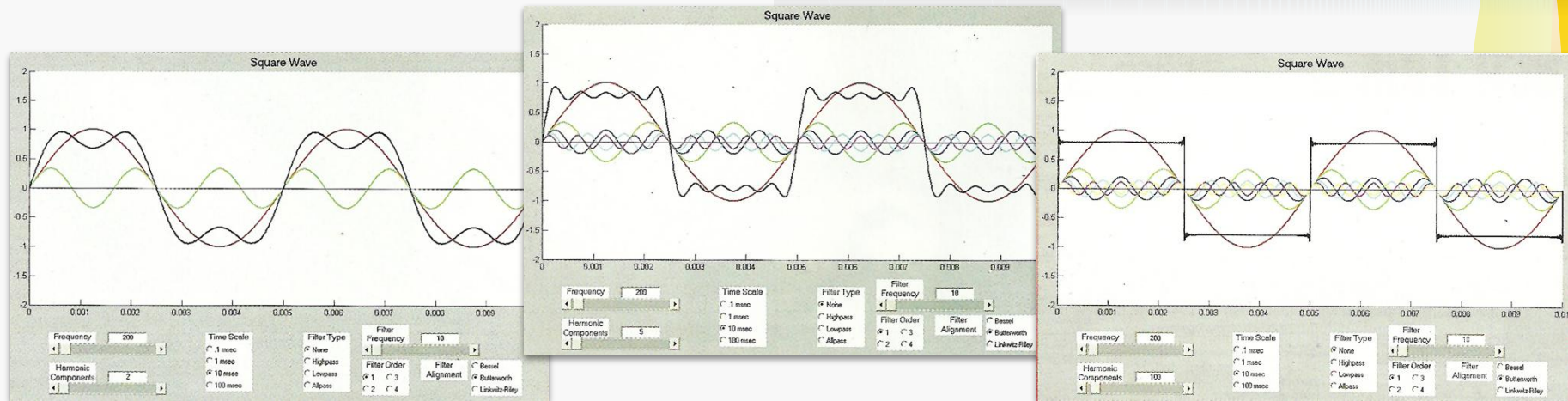
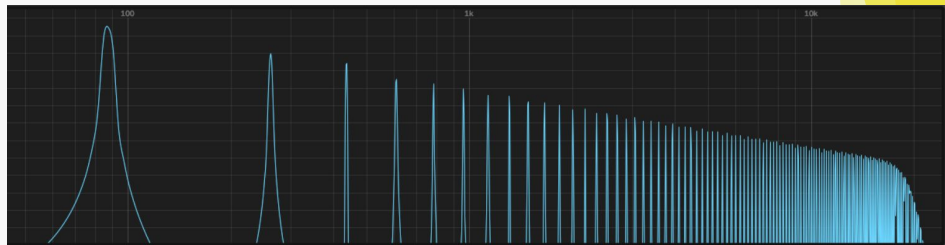
Un nuovo approccio: Sintesi Additiva

- Per il teorema di Fourier, qualunque funzione periodica di frequenza $f_0 = 1/T_0$, può essere rappresentata mediante una somma di funzioni **sinusoidali pure** di opportuna ampiezza e di frequenza multipla della frequenza fondamentale f_0 .
- Data una frequenza fondamentale f_0 , la **serie armonica** da essa generata è la successione dei suoi multipli interi ($f_0*1, f_0*2, f_0*3, \dots$). Queste frequenze multiple vengono dette **armoniche** o armonici.
- Uno **spettrogramma** si ottiene calcolando la trasformata di Fourier (solitamente FFT) su sezioni del segnale.
Lo spettrogramma permette di analizzare graficamente la serie armonica di un segnale.

Un nuovo approccio: Sintesi Additiva

○ Serie armonica Square Wave:

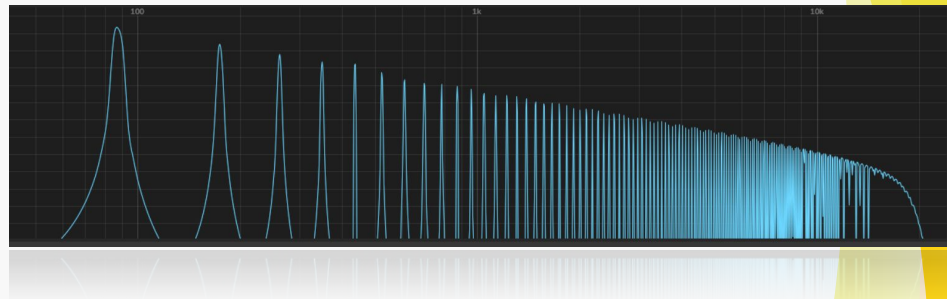
Una sinusoide fondamentale e **armoniche dispari**. L'ampiezza delle armoniche è uguale a $1/N$, dove N è l'armonica (1, 3, 5, 7...).



Un nuovo approccio: Sintesi Additiva

○ Serie armonica Sawtooth Wave:

Una sinusoide fondamentale e da tutte le **armoniche intere**. L'ampiezza delle armoniche è uguale a $1/N$, dove N è l'armonica (1, 2, 3, 4, 5, 6...).



$N = 0$



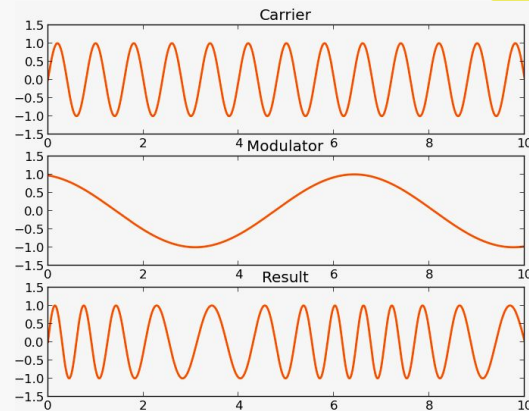
Sintesi Additiva

```
1 reference
public double calculateSignalValue(double newSignalTime, double newSignalFrequency)
{
    double signalSum = 0.0;
    for (int i = 0; i < subSignals.Length; i++)
    {
        signalSum += squareHarmonicLevels[i] * subSignals[i].calculateSignalValue(newSignalTime, newSignalFrequency * (i + 1));
    }
    return signalSum;
}
```

Modulazioni & Effetti

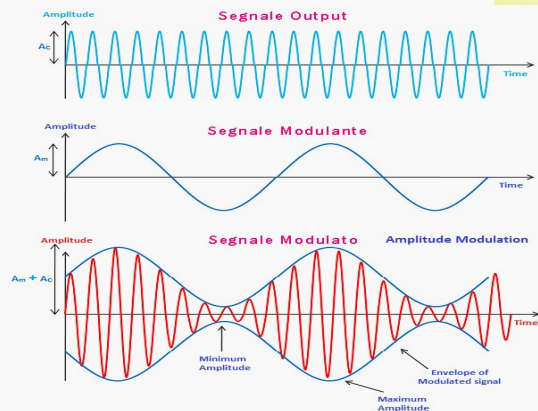
- **Frequency Modulation:** Moduliamo la **frequenza** dell'oscillatore, utilizzando un qualsiasi altro segnale (**segnale modulante**).

Ad ogni nuovo campionamento, verrà aggiunto un **offset** alla frequenza del segnale del synth.

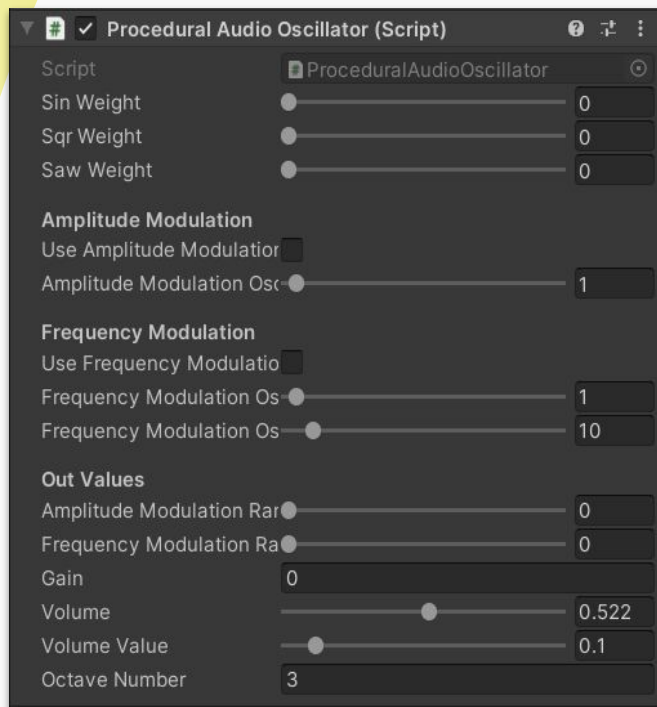


- **Amplitude Modulation:** Moduliamo l'**ampiezza** dell'oscillatore, utilizzando un qualsiasi altro segnale (**segnale modulante**).

Ad ogni nuovo campionamento, verrà moltiplicato il campionamento appena effettuato per l'ampiezza del segnale modulante in quell'istante.



Modulazioni & Effetti

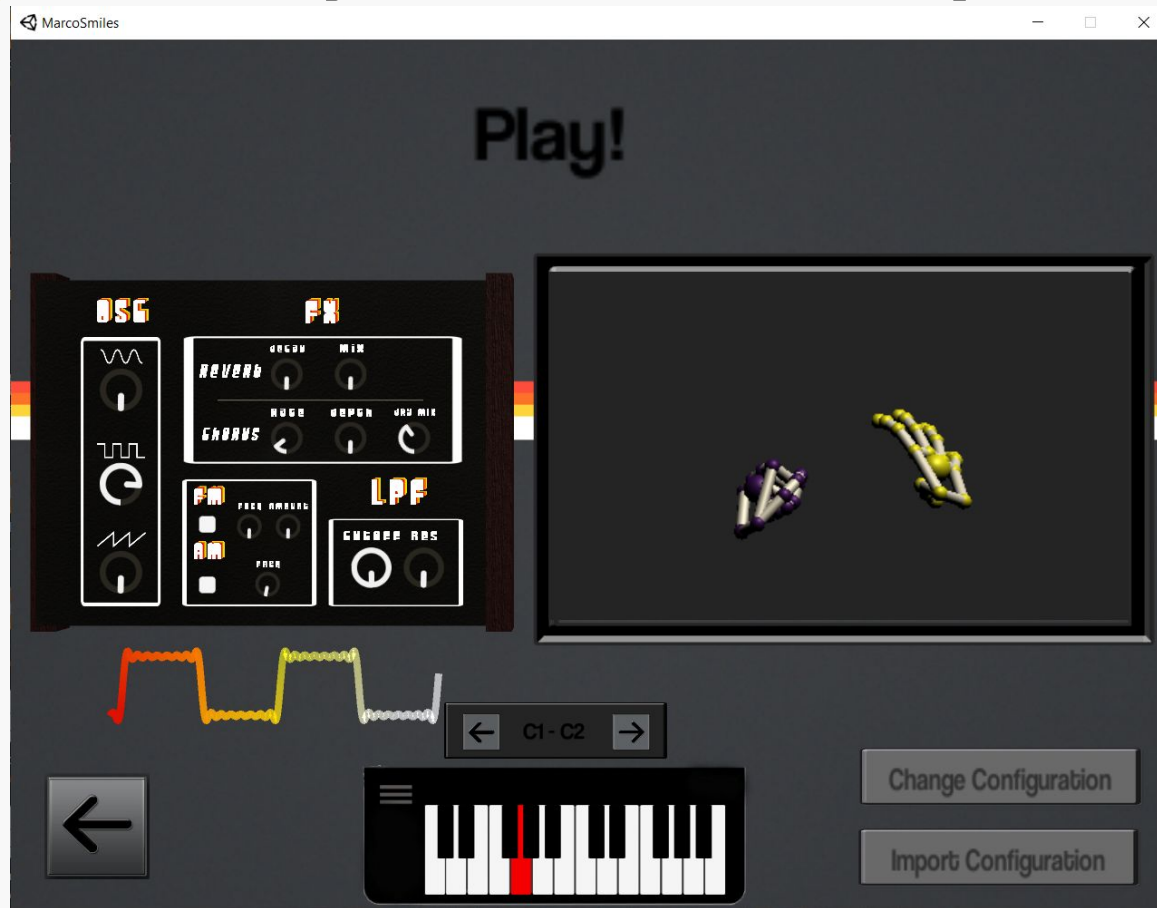


GUI & Oscilloscopio

- Possibilità di modificare parametri relativi al suono che si sta generando grazie ad **un'interfaccia grafica**.
Il design è totalmente ispirato a quello di un sintetizzatore reale.
Sono previste delle **manopole**, che simulano i potenziometri di un synth fisico.
- Per analizzare il suono che si sta generando, è stato implementato un **Oscilloscopio** che disegna la forma d'onda che il sintetizzatore e Unity stanno riproducendo.



GUI Synth & Oscilloscopio



Risultati & Discussioni

- La sintesi additiva, è risultata essere una tecnica efficace per generare forme d'onda complesse in modo semplice.
- Permette di aumentare notevolmente le possibili forme d'onda generabili.

In futuro:

- Aumento espressività del suono;
- Estensione forme d'onda disponibili;
- Aggiunta presets;
- Estensione Modulazioni;
- Interfacciamento con MIDI e DAW.





UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA



Grazie per l'attenzione



UNIVERSITÀ DEGLI STUDI
DI SALERNO